

TTS and STT Wrapper for virtual and physical robots enhanced by GenAI

מעטפת טקסט- לדיבור ודיבור- לטקסט לרובוטים וירטואליים ופיזיים משופרים
באמצעות בינה מלאכותית גנרטיבית

פרויקט גמר בפיתוח מערכות תוכנה
המכללה האקדמית רופין

מנחה אקדמית: ד"ר רינה צביאל- גירשין

פרויקט מספר: 9
שנת לימודים: תשפ"ו (2025-2026)

תוכן עניינים

- 2 -	תקציר מנהלים.....
- 3 -	הצגת הבעיה והצורך.....
- 3 -	הגדרת הבעיה.....
- 3 -	הצורך העסקי והחברתי.....
- 4 -	רקע ומצב קיים.....
- 4 -	סקירת טכנולוגיות קיימות.....
- 5 -	פתרונות קיימים ומגבלותיהם.....
- 5 -	הצורך בפתרון חדש.....
- 6 -	תיאור הפתרון המוצע.....
- 6 -	סקירה כללית.....
- 6 -	רכיבי המערכת.....
- 8 -	דרישות המערכת.....
- 8 -	דרישות פונקציונליות.....
- 9 -	דרישות לא פונקציונליות.....
- 10 -	ארכיטקטורה ותכנון.....
- 10 -	ארכיטקטורת המערכת.....
- 11 -	עקרונות תכנון.....
- 12 -	תכנון מסד הנתונים.....
- 13 -	מיומש והטמעה.....
- 13 -	טכנולוגיות וכלים.....
- 14 -	תהליך הפיתוח.....
- 15 -	אתגרים וחידושים.....
- 16 -	תוצאות והערכה.....
- 16 -	תוצרי הפרויקט.....
- 17 -	הערכת ביצועים.....
- 17 -	משוב ממשתמשים.....
- 18 -	מסקנות והמלצות.....
- 18 -	מסקנות.....
- 18 -	המלצות לפיתוח עתידי.....
- 19 -	סיכום.....
- 21 -	רשימה ביבליוגרפית.....
- 22 -	נספחים.....
- 22 -	נספח א': מבנה קבצי הפרויקט.....
- 24 -	נספח ב': הוראות התקנה.....
- 25 -	נספח ג': ממשקי API.....

תקציר מנהלים

פרויקט זה מציג פיתוח של מערכת Wrapper חדשנית לשילוב פונקציות טקסט-לדיבור (TTS) ודיבור-לטקסט (STT) ברובוטים וירטואליים ופיזיים, המונעים ע"י בינה מלאכותית גנרטיבית. המערכת נועדה ליצור ממשק גמיש ונגיש המאפשר תקשורת טבעית ואינטואיטיבית בין בני אדם לרובוטים.

הפרויקט כולל פיתוח מערכת תוכנה מבוססת ענן המשלבת טכנולוגיות מתקדמות של בינה מלאכותית, ביניהן OpenAI GPT לעיבוד שפה טבעית, OpenAI Whisper לזיהוי דיבור ו-OpenAI TTS להפקת דיבור. המערכת מספקת ממשק אחיד ופשוט לשימוש המאפשר למפתחים ולמשתמשי קצה לתקשר עם רובוטים באמצעות שפה טבעית, ללא צורך בהבנה מעמיקה של הטכנולוגיות הבסיסיות.

המערכת כוללת ממשק משתמש אינטראקטיבי מבוסס דפדפן, מערכת אימות משתמשים, פאנל ניהול למנהלים ותמיכה בשני מצבי שיחה: מצב רגיל (שימוש בהודעות כתובות) ומצב שיחה המאפשר דו-שיח רציף ללא צורך בלחיצות חוזרות (דמוי שיחה בין שני אנשים).

התוצר הסופי הוא מערכת מלאה ופועלת שיכולה לשמש כבסיס לפיתוח יישומים בתחומים כמו חינוך, בריאות ושירות הלקוחות.

הצגת הבעיה והצורך

הגדרת הבעיה

בעידן המודרני, רובוטים וסוכנים וירטואליים הופכים לחלק בלתי נפרד מחיי היומיום שלנו. עם זאת, האינטראקציה בין בני אדם לרובוטים מוגבלת בדרכי לממשקים גרפיים מסורתיים או לשפת תכנות מורכבת. הצורך באינטראקציה טבעית באמצעות שפה מדוברת הולך וגובר, במיוחד עבור משתמשים שאינם טכנולוגיים כגון ילדים, קשישים ובעלי מוגבלויות.

הבעיה המרכזית שהפריקט מטפל בה היא המורכבות הטכנית והפיצול שקיימים כיום בשילוב טכנולוגיות דיבור ברובוטים:

1. מורכבות טכנית: שילוב טכנולוגיות TTS ו-STT דורש ידע מעמיק בממשקי API שונים, פרוטוקולי תקשורת ועיבוד אותות.
2. העדר אחידות: קיימים ספקים שונים של שירותי דיבור כמו IBM, OpenAI ו-Google אשר כל אחד מהם עם ממשק ופורמט שונה.
3. קושי באינטגרציה: רובוטים פיזיים וכן וירטואליים דורשים גישות שונות לשילוב טכנולוגיות דיבור.
4. חוויית משתמש לא אופטימלית: רוב הפתרונות הקיימים אינם מספקים חווית שיחה טבעית ורציפה.

הצורך העסקי והחברתי

הצורך במערכת כזו נובע ממספר תחומים:

- חינוך: מורים וירטואליים המסוגלים לתקשר עם תלמידים בשפה טבעית ולהתאים את עצמם לצרכים אישיים.
- בריאות: עוזרים רובוטיים בבתי חולים ובבתים המסוגלים לשוחח עם מטופלים, לספק מידע ולהעניק תמיכה רגשית.
- שירות לקוחות: סוכנים וירטואליים המספקים שירות 24/7 בשפה טבעית.
- גישות: מתן גישה לטכנולוגיה לאנשים עם מוגבלויות שמתקשים בממשקים מסורתיים.
- בתים חכמים: שליטה בבית באמצעות שיחה טבעית עם מערכות חכמות.

רקע ומצב קיים

סקירת טכנולוגיות קיימות

¹בתחום טכנולוגיות הדיבור והבינה המלאכותית קיימות מספר טכנולוגיות מובילות:

ספק	טכנולוגיית STT	טכנולוגיית TTS	יתרונות וחסרונות
OpenAI	Whisper	OpenAI TTS	דיוק גבוה, תמיכה במודלים גנרטיביים, עלות גבוהה
Google Cloud	Speech-to-Text	Text-to-Speech	אמינות גבוהה, תמיכה בשפות רבות, תלות בענן
Amazon AWS	Transcribe	Polly	אינטגרציה עם AWS, מחיר תחרותי, מורכבות
IBM Watson	Speech to Text	Text to Speech	יכולות NLP מתקדמות, עלות גבוהה

¹ טבלה 1: השוואת טכנולוגיות TTS ו-STT.

פתרונות קיימים ומגבלותיהם

קיימים מספר פתרונות בשוק, אך כולם סובלים ממגבלות משמעותיות:

- חוסר אחידות: כל ספק מספק ממשק API שונה, מה שמחייב התאמה של הקוד לכל שירות.
- מורכבות טכנית: שילוב הטכנולוגיות דורש ידע מעמיק בתחום.
- העדר תמיכה רציפה: רוב הפתרונות דורשים הפעלה מחדש של הקלטה לכל משפט.
- חוויית משתמש לא אופטימלית: לא קיימת מערכת המשלבת את כל הרכיבים בצורה חלקה.
- עלות גבוהה: שימוש ישיר בשירותי הענן הינו יקר למפתחים קטנים.

הצורך בפתרון חדש

הפרויקט שלנו נועד למלא את הפערים הללו על ידי יצירת Wrapper אחיד ופשוט לשימוש המסתיר את המורכבות הטכנית ומספק חווית משתמש מיטבית. המטרה היא ליצור מערכת שכל מפתח יוכל להשתמש בה בקלות, תוך התמקדות בלוגיקה העסקית של האפליקציה שלו ולא בפרטים הטכניים של שירותי הדיבור.

תיאור הפתרון המוצע

סקירה כללית

הפתרון שפיתחנו הוא מערכת Wrapper מקיפה המשלבת:

- I. ממשק API אחיד: שכבת הפשטה המאחדת את כל שירותי ה-TTS וה-STT תחת ממשק אחד.
- II. מערכת ניהול משתמשים: מערכת אימות ואוטוריזציה מלאה עם תמיכה במנהלים ומשתמשים רגילים.
- III. ממשק משתמש אינטראקטיבי: אפליקציית דפדפן מודרנית ונגישה.
- IV. מצב שיחה (Conversation Mode): מצב ייחודי המאפשר דו שיח רציף וטבעי.
- V. אינטגרציה עם GPT: שימוש במודלים גנרטיביים לתגובות חכמות ומתואמות הקשר.
- VI. ארכיטקטורה מודולרית: עיצוב המאפשר החלפה קלה של ספקי שירות.

רכיבי המערכת

המערכת בנויה משלושה רכיבים ענקיים:

1. צד שרת (Backend) – ASP.NET

השרת מבוסס על ארכיטקטורת שלוש השכבות:

- a. שכבת בקרים (Controllers): מטפלת בבקשות HTTP ומנתבת אותן לשכבת הלוגיקה העסקית.
 - i. ChatController: מטפל בבקשות שיחה.
 - ii. SttController: מטפל בהמרת דיבור לטקסט.
 - iii. TtsController: מטפל בהמרת טקסט לדיבור.
 - iv. UserController: מטפל בניהול משתמשים.
- b. שכבת לוגיקה עסקית (Services): מכילה את הלוגיקה העסקית של האפליקציה.
 - i. ChatService: מנהל את השיחה עם ה-AI.
 - ii. SpeechService: מנהל את שירותי הדיבור.
 - iii. UserService: מנהל את המשתמשים.

c. שכבת גישה לנתונים (DAL – Data Access Layer): מתממשקת עם מקורות המידע.

i. OpenAiChatProvider: אינטגרציה עם GPT.

ii. OpenAiSpeechProvider: אינטגרציה עם Whisper ו-TTS.

iii. UserService: גישה למסד הנתונים.

2. צד הלקוח (Frontend) – JavaScript + HTML + CSS

הממשק מבוסס על ארכיטקטורה מודולרית:

a. Index.html: עמוד הבית והשיחה הראשי

b. app.js: לוגיקת האםליקציה הראשית

c. api.js: שכבת תקשורת עם השרת

d. script.js: כלים משותפים (VoiceInputController)

e. styles.css: עיצוב אחיד ומודרני

f. login.html/ register.html: דפי אימות

g. profile.html: דף ניהול פרופילים ומנהלים

3. מסד נתונים – SQL Server

מסד הנתונים מכיל טבלת משתמשים (NLA_Users) עם השדות הבאים:

a. UserId: מזהה ייחודי

b. Name: שם המשתמש

c. Email: כתובת דוא"ל (ייחודית)

d. Organization: ארגון (אופציונלי)

e. PasswordHash: סיסמה מוצפנת (SHA256)

f. IsAdmin: דגל מנהל

g. CreatedAt: תאריך יצירה

h. LastLoginAt: תאריך כניסה אחרון

i. IsActive: סטטוס חשבון

דרישות המערכת

דרישות פונקציונליות

הדרישות הפונקציונליות של המערכת כוללות:

- ניהול משתמשים:
 - רישום משתמש חדש עם אימות שדות
 - התחברות משתמש קיים
 - עדכון פרופיל אישי
 - מחיקת חשבון
 - התנתקות מהמערכת
- תקשורת עם AI:
 - שליחת הודעת טקסט ל- AI וקבלת תגובה
 - הקלטת הודעת דיבור והמרתה לטקסט
 - המרת תגובת ה- AI לדיבור
 - תצוגת היסטוריית שיחה
- מצב שיחה מתקדם
 - זיהוי אוטומטי של דיבור משתמש
 - זיהוי שתיקה ועצירה אוטומטית של הקלטה
 - השמעת תגובה אוטומטית
 - חזרה אוטומטית למצב האזנה
 - תמיכה בהקלדה ידנית בזמן מצב שיחה
- ניהול מערכת (למנהלים):
 - צפייה ברשימת כל המשתמשים
 - סינון וחיפוש משתמשים
 - קידום/ הורדה של משתמש לתפקיד מנהל
 - הפעלה/ השבתה של חשבונות
 - מחיקת משתמשים
 - צפייה בסטטיסטיקות מערכת

דרישות לא פונקציונליות

ביצועים:

- זמן תגובה מקסימלי: 3 שניות להודעה רגילה
- זמן המרת STT: עד 2 שניות
- זמן הפקת TTS: עד 2 שניות
- תמיכה בלפחות 10 משתמשים במקביל

אבטחה:

- הצפנת סיסמאות עם SHA256
- אימות משתמש לכל בקשה
- הגנה מפני SQL Injection
- תקשורת מוצפנת (HTTPS מומלץ)
- הפרדה בין הרשאות מנהל למשתמש רגיל

שמישות (Usability):

- ממשק משתמש אינטואיטיבי ונגיש
- תמיכה בעברית ובאנגלית
- עיצוב רספונסיבי למכשירים שונים
- הודעות שגיאה ברורות
- משוב ויזואלי למשתמש

תחזוקה:

- ארכיטקטורה מודולרית להחלפת רכיבים
- קוד מתועד ונקי
- לוגים מפורטים לניפוי באגים
- אפשרות להחלפת ספק AI בקלות

זמינות:

- זמינות של 99% בשעות פעילות
- גיבויים אוטומטיים למסד הנתונים
- טיפול בשגיאות והתאוששות אוטומטית

ארכיטקטורה ותכנון

ארכיטקטורת המערכת

המערכת מבוססת על ארכיטקטורת Client- Server עם הפרדה ברורה בין הרכיבים:

[דפדפן/ לקוח]



[שכבת ממשק - HTML/ CSS/ JS]

- app.js (לוגיקה ראשית)
- api.js (תקשורת)
- script.js (כלים משותפים)

[HTTPS/ REST API]



[שרת ASP.NET Core]

- Controllers (בקרים)
 - ChatController
 - SttController
 - TtsController
 - UserController
- Services (לוגיקה עסקית)
 - ChatService
 - SpeechService
 - UserService
- DAL (גישה לנתונים)
 - Providers
 - OpenAiChatProvider

OpenAiSpeechProvider ▪
Database Access ○



[מסד נתונים – SQL Server]

[שירותי OpenAI – GPT, Whisper, TTS]

עקרונות תכנון

התכנון מבוסס על עקרונות הנדסת תוכנה מובילים:

1. Separation of Concerns כל רכיב אחראי על תפקיד ספציפי בלבד:

a. Controllers: ניתוב בקשות

b. Services: לוגיקה עסקית

c. Providers: אינטגרציה חיצונית

d. DAL: גישה לנתונים

2. Dependency Injection שימוש ב-DI לניהול תלויות ולהקלת בדיקות:

a. `builder.Services.AddScoped<IAiProvider, OpenAiChatProvider>();`

b. `builder.Services.AddScoped<ISpeechProvider,`

`OpenAiSpeechProvider>();`

c. `builder.Services.AddScoped<IChatService, ChatService>();`

3. Interface-Based Design הגדרת ממשקים לכל שירות:

a. `IAiProvider`: ממשק לספקי AI

b. `ISpeechProvider`: ממשק לשירותי דיבור

c. `IUserService`: ממשק לניהול משתמשים

4. Single Responsibility Principle כל מחלקה אחראית על פונקציונליות אחת בלבד.

5. Open/ Closed Principle המערכת פתוחה להרחבה אך סגורה לשינוי- ניתן להוסיף

ספקי AI חדשים ללא שינוי קוד קיים.

תכנון מסד הנתונים

מסד הנתונים מכיל טבלה אחת עיקרית- NLA_Users:

שדה	סוג	תיאור	אילוצים
UserId	NVARCHAR(50)	מזהה ייחודי	PRIMARY KEY
Name	NVARCHAR(100)	שם המשתמש	NOT NULL
Email	NVARCHAR(225)	כתובת דוא"ל	UNIQUE, NOT NULL
Organization	NVARCHAR(200)	ארגון	NULL
PasswordHash	NVARCHAR(255)	סיסמה מוצפנת	NOT NULL
IsAdmin	BIT	דגל מנהל	DEFAULT 0
CreatedAt	DATETIME2	תאריך יצירה	DEFAULT GETUTCDATE()
LastLoginAt	DATETIME2	כניסה אחרונה	NULL
IsActive	BIT	סטטוס פעיל	DEFAULT 1

² טבלה 2: מבנה טבלת משתמשים

מימוש והטמעה

טכנולוגיות וכלים

:Backend

- ASP.NET Core 8.0 פלטפורמת השרת
- C# שפת התכנות
- SQL Server מסד הנתונים
- Entity Framework Core – ORM (אופציונלי)
- Microsoft.Data.SqlClient גישה למסד הנתונים

:Frontend

- HTML5 מבנה הדפים
- CSS3 עיצוב ואנימציות
- JavaScript לוגיקה
- Web Audio API הקלטה ושמע
- MediaRecorder API הקלטת קול
- Fetch API תקשורת עם השרת

שירותי צד שלישי:

- OpenAI GPT-3.5-turbo שיחה חכמה
- OpenAI Whisper המרת דיבור לטקסט
- OpenAI TTS המרת טקסט לדיבור

כלי פיתוח:

- Visual Studio Code
- SQL Server Management Studio
- Git ניהול גרסאות
- Postman בדיקת API

תהליך הפיתוח

הפיתוח בוצע בשלבים הבאים:

1. תכנון ואפיון

- a. הגדרת דרישות
- b. עיצוב ארכיטקטורה
- c. תכנון מסד נתונים
- d. בחירת טכנולוגיות

2. פיתוח Backend

- a. הקמת פרויקט ASP.NET
- b. יצירת מבנה שכבות
- c. מימוש Providers (Real + Mock)
- d. פיתוח Services
- e. פיתוח Controllers
- f. אינטגרציה עם OpenAI

3. פיתוח Frontend

- a. עיצוב UX/UI
- b. פיתוח ממשק משתמש
- c. מימוש לוגיקת שיחה
- d. פיתוח VoiceInputController
- e. מימוש Conversation Mode
- f. אינטגרציה עם Backend

4. מערכת ניהול משתמשים

- a. פיתוח מערכת אימות
- b. דפי רישום והתחברות
- c. דף ניהול פרופיל
- d. פאנל מנהלים

5. בדיקות ונתונים

- a. Unit Tests
- b. User Acceptance Testing

c. תיקון באגים

6. תיעוד והדרכה

a. תיעוד קוד

b. מדריך מפתח

c. מדריך משתמש

אתגרים וחידושים

אתגר 1: זיהוי שתיקה ב- Conversation Mode

פתרון: שילוב של Voice Activity Detection באמצעות Web Audio API עם ניתוח

תדירויות אודיו בזמן אמת.

אתגר 2: חווית משתמש חלקה

פתרון: מערכת משוב ויזואלית מפורטת עם אנימציות ואינדיקטורים ברורים.

אתגר 3: ביצועים של שירותי OpenAI

פתרון: שימוש במודל GPT-3.5-turbo (מהיר וזול יחסית) במקום GPT 4.

חידוש: Conversation Mode. פיתחנו מצב שיחה ייחודי המאפשר דו-שיח רציף ללא צורך

בלחיצות, עם זיהוי אוטומטי של תחילת ותום דיבור.

תוצאות והערכה

תוצרי הפרויקט

הפרויקט הניב את התוצרים הבאים:

1. מערכת תוכנה מלאה ופועלת הכוללת:

- a. שרת Backend מבוסס ASP.NET Core
- b. ממשק Frontend מודרני
- c. מסד נתונים SQL Server
- d. אינטגרציה מלאה עם שירותי OpenAI

2. ממשק משתמש אינטראקטיבי הכולל:

- a. דף שיחה ראשי
- b. מצב רגיל להתכתבות בהודעות
- c. Conversation Mode לניהול שיחה
- d. פאנל ניהול למנהלים
- e. דפי אימות מעוצבים

3. תיעוד מקיף:

- a. תיעוד קוד מפורט
- b. מדריך התקנה
- c. מדריך משתמש

4. קוד מקור:

- a. 29 קבצי קוד
- b. מעל 3,000 שורות קוד
- c. ארכיטקטורה מודולרית ונקייה

הערכת ביצועים

ביצענו מדידות ביצועים למרכיבים שונים³:

הערות	זמן ממוצע	פעולה
תלוי באורך ההודעה	2-3 שניות	שליחת הודעה וקבלת תגובה
תלוי באורך ההקלטה	1-2 שניות	המרת דיבור לטקסט (STT)
תלוי באורך הטקסט	1-2 שניות	המרת טקסט לדיבור (TTS)
כולל הצפנת סיסמה	< 1 שנייה	רישום משתמש
אימות וטעינת נתונים	< 1 שנייה	התחברות

משוב ממשתמשים

נקודות חוזק שזוהו:

- ממשק משתמש אינטואיטיבי וקל לשימוש
- Conversation Mode מספק חווית שיחה טבעית
- תגובות ה-AI איכותיות ורלוונטיות
- עיצוב מודרני ומושך
- ביצועים טובים

תחומים לשיפור:

- שיפור זיהוי שתיקה בסביבות רועשות
- הוספת היסטוריית שיחות קודמות
- תמיכה בצירוף קבצים

³ טבלה 3: ביצועי המערכת

- אפשרות להתאמה אישית של קול ה-AI

מסקנות והמלצות

מסקנות

הפרויקט השיג את מטרותיו והצליח ליצור מערכת Wrapper פונקציונלית ומתקדמת לשילוב טכנולוגיות TTS ו-STT ברובוטים. המערכת מספקת משק אחיד ופשוט לשימוש, מסתירה את המורכבות הטכנית ומאפשרת למפתחים להתמקד בלוגיקה העסקית של האפליקציה שלהם.

עמידה ביעדי הפרויקט:

- פיתוח מערכת Wrapper לטכנולוגיות TTS ו-STT
- יצירת ממשק משתמש פשוט ונגיש
- שיפור חווית השיחה והתקשורת
- תמיכה ביכולות שמיעה ודיבור
- שימוש במודלים גנרטיביים של AI
- כלים מותאמים אישית להפקת דיבור
- אפשרות ליישום בתחומים שונים

תובנות שנלמדו:

- חשיבות הפשטות: ממשק API פשוט ואינטואיטיבי הוא קריטי להצלחה.
- חווית משתמש היא המפתח: השקעה בעיצוב UX איכותי מחזירה את עצמה בשביעות רצון המשתמשים.
- מודלריות משתלמת: ארכיטקטורה מודולרית מקלה על תחזוקה והרחבה.
- בדיקות מקיפות חשובות: בדיקות בשלבים מוקדמים חוסכות זמן מאוחר יותר.
- תיעוד הוא חובה: תיעוד טוב מקל על העברת ידע ותחזוקה עתידית.

המלצות לפיתוח עתידי

שיפורים קצרי טווח (3-6 חודשים):

1. היסטוריית שיחות: שמירת שיחות קודמות ואפשרות לחזור אליהן.
2. התאמה אישית: מתן אפשרות למשתמש לבחור קול, מהירות דיבור ואישיות AI.
3. שיפור Conversation Mode: זיהוי טוב יותר של שתיקה בסביבות רועשות.
4. אפליקציה ניידת: פיתוח אפליקציה ל- iOS ו- Android.

הרחבות ארוכות טווח (6-12 חודשים):

1. אינטגרציה עם רובוטים פיזיים: פיתוח SDK לחיבור רובוטים פיזיים למערכת.
2. מערכת המלצות: AI שלומד את העדפות המשתמש ומציע תוכן רלוונטי.
3. אנליטיקה: מערכת ניתוח שימוש ומדדי ביצועים.

יישומים עתידיים אפשריים:

- חינוך: עוזר וירטואלי ללימוד שפות, מתמטיקה ומקצועות נוספים.
- בריאות: מלווה רפואי לניטור מטופלים ומתן תזכורות.
- שירות לקוחות: צ'אט-בוט חכם לתמיכה 24/7.

פרויקט זה הראה כי ניתן ליצור מערכת Wrapper יעילה ופשוטה לשימוש המאפשרת תקשורת טבעית בין בני אדם לרובוטים. המערכת שפיתחנו מספקת בסיס איתן לפיתוח יישומים מתקדמים בתחומים שונים ומוכיחה את הפוטנציאל העצום של שילוב AI עם ממשקי דיבור. אנו מאמינים שמערכת זו יכולה לשמש כנקודת פתיחה לפיתוחים עתידיים ולתרום להנגשת טכנולוגיות AI לקהל רחב יותר. מעבר להישג הטכני הפרויקט מדגיש מספר תובנות רחב:

1. דמוקרטיזציה של טכנולוגיות דיבור: החסם העיקרי לאימוץ טכנולוגיות קוליות היה עד כה המורכבות הטכנית והפרגמנטציה בין ספקים. ה- Wrapper שפותח בפרויקט זה מוכיח כי ניתן להסיר חסמים אלו וליצור סטנדרטיזציה שמאפשרת גם למפתחים ללא רקע בעיבוד אותות ליצור חוויות משתמש קוליות עשירות.
 2. המעבר מממשק ויזואלי (GUI) לממשק קולי (VUI): Conversation Mode והאינטגרציה עם מודלי שפה (LLMs) ממחישה את המעבר הפרדיגמטי בתעשייה- ממסכים וכפתורים לאינטראקציה טבעית ובלתי אמצעית. הפרויקט הראה כיצד שילוב של זיהוי שתיקה חכם וזמני תגובה מהירים הוא המפתח ליצירת אשליה של שיחה אנושית זורמת.
 3. ארכיטקטורה צופה פני עתיד: הבחירה בארכיטקטורה מודולרית (הפרדת Services ו- Providers) מבטיחה כי המערכת תישאר רלוונטית גם כשיצאו מודלים חדשים של AI. היכולת להחליף את "מוח" המערכת (GPT) או את "פה" "ואוזני" המערכת (TTS/ STT) ללא שכתוב של הלוגיקה העסקית, היא הערך המוסף האמיתי של פתרון זה לטווח ארוך.
- לסיכום, המערכת מהווה אבן דרך בחיבור בין העולם הפיזי (רובוטים) לעולם הווירטואלי (GenAI) ומספקת פלטפורמה נגישה ליצירת הדור הבא של עוזרים חכמים.

- OpenAI. (2024). OpenAI API Documentation.
<https://platform.openai.com/docs>
- Microsoft. (2024). ASP.NET Core Documentation.
<https://docs.microsoft.com/aspnet/core>
- Mozilla Developer Network. (2024). Web APIs - MediaRecorder.
<https://developer.mozilla.org/en-US/docs/Web/API/MediaRecorder>
- Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
- Nielsen, J. (1995). 10 Usability Heuristics for User Interface Design. Nielsen Norman Group.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. (2008). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- Fowler, M. (2018). Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley.
- Brown, W., Malveau, R., McCormick, H., & Mowbray, T. (1998). AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis. Wiley.

נספחים

נספח א': מבנה קבצי הפרויקט

:Frontend Files

- index.html – עמוד הבית ושיחה
- login.html – דף התחברות
- register.html – דף רישום
- profile.html – דף ניהול פרופיל
- app.js – לוגיקת האפליקציה
- api.js – תקשורת עם השרת
- script.js – כלים משותפים
- styles.css – עיצוב
- config.js – הגדרות

:Backend Files

- Program.cs – נקודת כניסה
- :Controllers
 - ChatContoller.cs
 - SttController.cs
 - TtsController.cs
 - UserController.cs
- :BL
 - ChatService.cs
 - SpeechService.cs
 - User.cs
- :DAL
 - IAiProvider.cs
 - ISpeechProvider.cs
 - OpenAiChatProvider.cs

OpenAiSpeechProvider.cs ○

MockAiProvider.cs ○

MockSpeechProvider.cs ○

:Configuration

• appsettings.json – הגדרות ומחרוזות חיבור

• UserTable.sql – סקריפט יצירת טבלה

נספח ב': הוראות התקנה

דרישות מערכת:

- NET 8.0 SDK
- SQL Server 2019 ומעלה
- Visual Studio Code
- דפדפן מודרני

שלבי התקנה:

1. שחזור מסד נתונים:

- a. לפתוח SQL Server Management
- b. להריץ את UserTable.sql
- c. לעדכן את Connection String ב- appsettings.json

2. הגדרת OpenAI API:

- a. להשיג את ה-API Key מ- <https://platform.openai.com>
- b. לעדכן את המפתח ב- appsettings.json

3. הרצת השרת:

- a. לפתוח את הפרויקט ב- VS Code
- b. ללחוץ F5 או בטרמינל dotnet run
- c. השרת יעלה ב- <http://localhost:5000>

4. הרצת הלקוח:

- a. לפתוח את index.html בדפדפן
- b. להשתמש ב- Live Server ב- VS Code

5. יצירת משתמש ראשון:

- a. לעבור ל- register.html
- b. למלא את הפרטים המתאימים
- c. התחברות עם פרטי המשתמש החדש דרך login.html

Chat API:

POST /api/chat

Body: { "message": "שלום", "userId": "..." }

Response: { "reply": "במה אוכל לעזור?" }

STT API:

POST /api/stt

Form-data: audio=<audio file>

Response: { "transcript": "הטקסט שהומר" }

TTS API:

POST /api/tts

Body: { "text": "שלום עולם" }

Response: <audio file>

User Registration:

POST /api/user/register

Body: { "name": "...", "email": "...", "password": "..." }

Response: { "userId": "...", "name": "...", ... }

User Login:

POST /api/user/login

Body: { "email": "...", "password": "..." }

Response: { "userId": "...", "name": "...", "isAdmin":
false, ... }

Update Profile:

PUT /api/user/{userId}

Body: { "name": "...", "organization": "...", "password":
"..." }

Response: { "userId": "...", "name": "...", ... }

Get All Users (Admin):

GET /api/user

Response: [{ "userId": "...", "name": "...", ... }, ...]